

1. What is CSS?

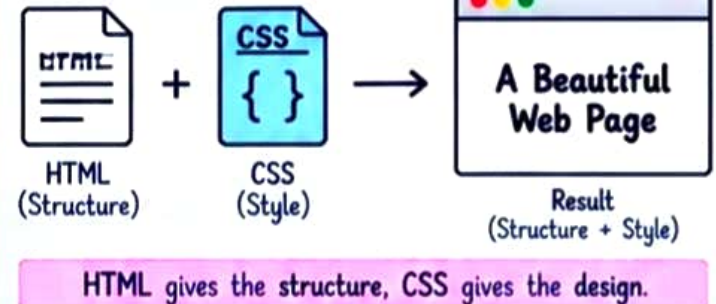
Introduction + History + Why CSS is Used + CSS3

Learn
Build
Grow
with
Codekit_

What is CSS ?

- CSS (Cascading Style Sheets) is a styling language used to design and layout web pages.
- It controls the look and feel of HTML elements like colors, fonts, spacing, layout, etc.
- CSS makes websites more attractive, user-friendly and responsive.
- It works with HTML to create modern and beautiful websites.

How CSS Works ?



Example

HTML (Structure)	CSS (Style)	Output (Result)
<pre><h1 class="title"> Hello World </h1> <p class="text"> This is a simple paragraph. </p></pre>	<pre>.title { color: #007bff; font-size: 32px; font-weight: bold; } .text { color: #333; font-size: 16px; }</pre>	

"Same HTML
Different Look
with CSS!"

↖ CSS turns
a simple page
into an attractive
website.

Brief History of CSS

- 1994** CSS was first proposed by Håkon Wium Lie.
- 1996** CSS1 was released by W3C (basic styling features).
- 1998** CSS2 was released with more layout and positioning features.
- 2011** CSS3 started to be developed (modules instead of a single version).
- 2023+** CSS3 is widely used with modern web features like Flexbox, Grid, Animations, etc.

Why is CSS Used ?

- ✓ Separates content (HTML) from design (CSS).
- ✓ Makes websites visually attractive.
- ✓ Helps in responsive design (works on mobile, tablet, desktop).
- ✓ Improves user experience (better readability and layout).
- ✓ Reduces code duplication.
- ✓ Easy to maintain and update styles.
- ✓ Allows advanced layouts and animations.

What is CSS3 ?

- CSS3 is the latest version of CSS.
- It introduces many new features and improvements.
- It is divided into modules (e.g., Flexbox, Grid, Animations, Transitions, Media Queries, etc.).
- CSS3 makes modern web design easier and more powerful.
- It supports responsive, mobile-friendly and interactive websites without extra plugins (like Flash).

Key Takeaways

- ✓ CSS is used to style and design web pages.
- ✓ It works with HTML.
- ✓ CSS has evolved from CSS1 → CSS2 → CSS3.
- ✓ CSS3 provides powerful modern features.
- ✓ A well-styled website looks better and improves user experience.

Tip:

Think of HTML as the skeleton and CSS as the clothes that make your website look amazing!

💡 Good
Design
Better
Experience



2. How to Add CSS to HTML

Inline CSS + Internal CSS + External CSS

Why Add CSS to HTML ?

- HTML gives structure to a web page, but it looks plain.
- CSS is used to style HTML and make it visually attractive.
- There are 3 ways to add CSS to HTML: Inline, Internal, and External.
- You can choose any method based on your project size and need.

Three Ways to Add CSS

-  **1** **Inline CSS**
(Inside HTML tag)
-  **2** **Internal CSS**
(Inside <head>)
-  **3** **External CSS**
(In a separate file)

Same Result Different Methods Choose What Fits You!

1 Inline CSS

CSS inside an HTML element using the style attribute.

HTML Code

```
<h1 style="color: blue; font-size: 32px;">
  Hello World!
</h1>

<p style="color: gray;">
  This is a paragraph.
</p>
```

Output (In Browser)

Hello World!

This is a paragraph.

Use Case:

- Quick styling
- For small changes

2 Internal CSS

CSS inside the <head> section using <style> tag.

HTML Code

```
<!DOCTYPE html>
<html>
<head>
  <title>Internal CSS</title>
  <style>
    h1 {
      color: green;
      font-size: 32px;
    }
    p {
      color: gray;
    }
  </style>
</head>
<body>
  <h1>Hello World!</h1>
  <p>This is a paragraph.</p>
</body>
</html>
```

Output (In Browser)

Hello World!

This is a paragraph.

Use Case:

- Useful for single page websites
- Keeps CSS and HTML in one file

3 External CSS

CSS in a separate .css file and linked using <link> tag.

HTML Code (index.html)

```
<!DOCTYPE html>
<html>
<head>
  <title>External CSS</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h1>Hello World!</h1>
  <p>This is a paragraph.</p>
</body>
</html>
```

CSS Code (style.css)

```
h1 {
  color: purple;
  font-size: 32px;
}
p {
  color: gray;
}
```

Output (In Browser)

Hello World!

This is a paragraph.

Use Case:

- Best for large websites
- Keeps HTML and CSS separate
- Easier to maintain

Key Takeaways

- ✓ CSS can be added in 3 ways: Inline, Internal, External.
- ✓ Inline CSS is quick but not recommended for large projects.
- ✓ Internal CSS is good for single pages.
- ✓ External CSS is the best practice for clean and maintainable code.
- ✓ All three methods give the same result.

Tip:

For real projects, always use External CSS. It keeps your code clean and organized!



3. CSS Syntax & Selectors

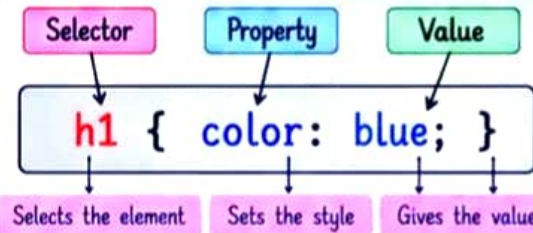
Selector + Property + Value + Element, Class & ID Selectors

Learn
Build
Grow
with
Codekit_

What is CSS Syntax ?

- CSS syntax is the set of rules for writing CSS code.
- It tells the browser which element to style and how to style it.
- A CSS rule consists of a selector, a property and a value.
- Each declaration ends with a semicolon (;).

CSS Syntax Structure



Selects the element Sets the style Gives the value

You can add multiple properties inside one selector.

Same
Syntax
Different
Selectors
More
Possibilities!

Types of CSS Selectors

1 Element Selector

Selects all elements of a specific tag.

CSS Code

```
h1 {
  color: blue;
  font-size: 32px;
}
```

HTML Example

```
<h1>This is a heading</h1>
```

Output (In Browser)

This is a heading

Use Case:

Style all heading, paragraph, div etc. elements.

2 Class Selector

Selects elements with a specific class attribute.

CSS Code

```
.title {
  color: green;
  font-size: 20px;
}
```

HTML Example

```
<h1 class="title">Hello  
World!</h1>
```

Output (In Browser)

Hello World!

Use Case:

Used for multiple elements with the same style.

3 ID Selector

Selects a single element with a unique id attribute.

CSS Code

```
#header {
  background-color: #007bff;
  color: white;
  padding: 10px;
}
```

HTML Example

```
<div id="header">Welcome  
to Codekit_</div>
```

Output (In Browser)

Welcome to Codekit_

Use Case:

Used for a single, unique element on a page.

4 Universal Selector

Selects all elements on the page.

CSS Code

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}
```

HTML Example

```
<p>All elements will have  
this style</p>
```

Output (In Browser)

All elements will have this style

Use Case:

Useful for applying common styles like reset.

A selector targets the HTML element you want to style.

More CSS Selectors (Quick View)

Selector	Example	Targets
Element	p	All <p> elements
Class	.btn	All elements with class="btn"
ID	#main	Single element with id="main"
Universal	*	All elements
Group	h1, p	Multiple elements
Descendant	div p	<p> inside <div>
Child	ul > li	Direct inside
Attribute	input[type="text"]	Elements with a specific attribute

Key Takeaways

- ✓ CSS syntax consists of selector, property and value.
- ✓ Element selector targets all elements of a tag.
- ✓ Class selector is used for multiple elements.
- ✓ ID selector is used for a single unique element.
- ✓ Universal selector targets all elements.
- ✓ There are many other selectors to target elements in different ways.
- ✓ Using the right selector makes your code clean and maintainable.

Tip:

Use meaningful class and id names for better readability and maintainability.

Clean
Selectors
Better
Websites!

Codekit_

"Right Selectors. Better Styles. Brighter Web Pages."



Like



Comment



Share



Save

4. CSS Colors & Backgrounds

Color Names + HEX + RGB + Background Color + Background Image

Why Colors & Backgrounds ?

- Colors make your website visually attractive and engaging.
- Backgrounds help highlight content and improve readability.
- With CSS, you can use color names, HEX, RGB and even images as backgrounds.
- Colors and backgrounds play a big role in UI/UX and modern web design.

Ways to Define Colors in CSS

1 Color Names

Use predefined color names.

```
h1 {
  color: blue;
}
```

Hello World

2 HEX Value

Use hexadecimal values.

```
h1 {
  color: #ff5733;
}
```

Hello World

3 RGB Value

Use RGB values.

```
h1 {
  color: rgb(0, 128, 0);
}
```

Hello World

4 RGBA Value

Use RGBA for transparency.

```
h1 {
  color: rgba(255, 0, 0, 0.5);
}
```

Hello World

1 Common Color Names

CSS provides many predefined color names.



2 HEX Colors

HEX colors are written with # followed by 6 characters (0-9, A-F).

#ff0000	Red
#00ff00	Green
#0000ff	Blue
#ffff00	Yellow
#ff5733	Orange
#9c27b0	Purple

3 RGB Colors

RGB colors use a combination of Red, Green and Blue (0 - 255).

rgb(255, 0, 0)	Red
rgb(0, 255, 0)	Green
rgb(0, 0, 255)	Blue
rgb(255, 255, 0)	Yellow
rgb(255, 165, 0)	Orange
rgb(123, 104, 238)	Medium Slate Blue

4 Background Color

Use `background-color` property to set a background color.

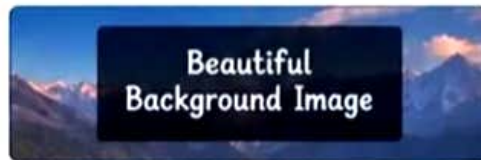
```
body {
  background-color: #f0f8ff;
}
```

This is a section with background color.

5 Background Image

Use `background-image` to add an image as a background.

```
body {
  background-image: url('bg.jpg');
  background-size: cover;
  background-repeat: no-repeat;
  background-position: center;
}
```



6 Background Properties

Other useful background properties.

Property	Description
<code>background-color</code>	Sets the background color
<code>background-image</code>	Adds a background image
<code>background-repeat</code>	Controls image repeat (no-repeat, repeat, etc.)
<code>background-size</code>	Sets the size (cover, contain, etc.)
<code>background-position</code>	Sets the image position (top, center, bottom, etc.)
<code>background-attachment</code>	Fixed or scroll with the page

Key Takeaways

- ✓ You can use color names, HEX, RGB and RGBA in CSS.
- ✓ `background-color` changes the background color.
- ✓ `background-image` adds an image as background.
- ✓ Use other background properties to control image behaviour.
- ✓ Colors and backgrounds improve the look and feel of your website.

Tip:

Choose colors that match your brand and keep good contrast for better readability and accessibility.



Like



Comment



Share



Save

5. CSS Text & Fonts

Font Family + Font Size + Font Weight + Text Align + Text Decoration

Learn
Build
Grow
with
Codekit_

1 Font Family

The font-family property sets the typeface for text.

CSS Code

```
p {
  font-family: Arial, "Segoe UI",
  Tahoma, sans-serif;
}
```

Output

This is a paragraph
with a custom font.

If the first font is not available,
the next one will be used
(fallback).

2 Font Size

The font-size property sets the size of the text.

CSS Code

```
h1 { font-size: 32px; }
p { font-size: 16px; }
.small { font-size: 12px; }
.large { font-size: 24px; }
```

Output

This is 32px

This is 16px

This is 12px

This is 24px

Size
Matters
Make It
Readable!

3 Font Weight

The font-weight property sets how bold or light the text is.

CSS Code

```
.light { font-weight: 300; }
.normal { font-weight: 400; }
.bold { font-weight: 700; }
.bolder { font-weight: 900; }
```

Output

Light Text (300)

Normal Text (400)

Bold Text (700)

Bolder Text (900)

4 Text Align

The text-align property sets the horizontal alignment of text inside an element.

CSS Code

```
.left { text-align: left; }
.center { text-align: center; }
.right { text-align: right; }
.justify { text-align: justify; }
```

Output

Left Aligned Text

Center Aligned Text

Right Aligned Text

This is a justified text. It looks
neat and clean, especially in
paragraphs.

5 Text Decoration

The text-decoration property is used to add lines to text.

CSS Code

```
.underline { text-decoration: underline; }
.overline { text-decoration: overline; }
.line-through { text-decoration: line-through; }
.none { text-decoration: none; }
```

Output

Underlined Text

Overline Text

~~Line Through Text~~

No Decoration

6 Google Fonts (Web Fonts)

You can also use online fonts from Google Fonts.

HTML (In <head>)

```
<link href="https://
fonts.googleapis.com/css2?
family=Poppins:wght@400;700&
display=swap" rel="stylesheet">
```

CSS Code

```
body {
  font-family: 'Poppins',
  sans-serif;
}
```

Output

Poppins Font

This text is using
Google Fonts (Poppins).

Modern Fonts
Make Your Website
Look Professional!

Key Takeaways

- ✓ font-family sets the typeface.
- ✓ font-size controls the text size.
- ✓ font-weight makes text light or bold.
- ✓ text-align aligns text (left, center, right, justify).
- ✓ text-decoration adds lines (underline, overline, line-through, none).
- ✓ Use Google Fonts for beautiful web typography.

Tip:

Use web-safe fonts or Google Fonts,
keep good contrast, and maintain
proper text size for better readability
and user experience.

Good
Typography
Better
Websites!



6. CSS Box Model

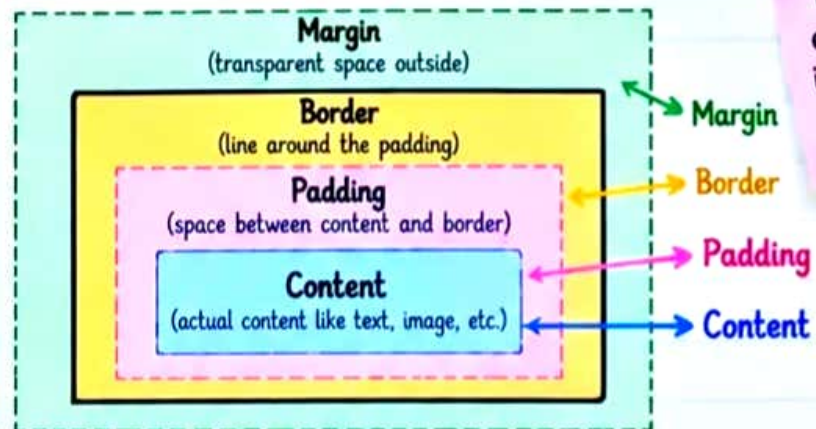
Content + Padding + Border + Margin

Learn
Build
Grow
with
Codekit_

What is the CSS Box Model ?

- In CSS, every element is treated as a rectangular box.
- The box model describes the total space an element takes on a web page.
- It consists of 4 parts: Content, Padding, Border and Margin.
- Understanding the box model helps you control the size, spacing and layout of elements.

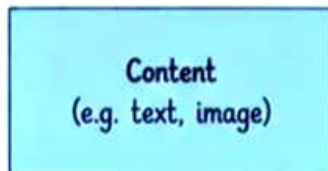
Box Model Diagram



Every
element
in CSS
is a Box!

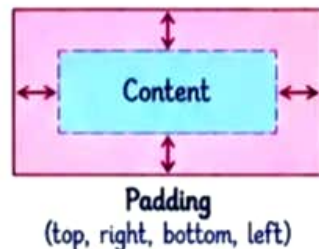
1 Content

The content is the actual area where your text, image, or other media appears.



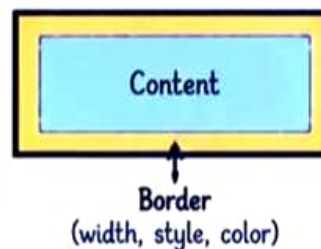
2 Padding

Padding is the space between the content and the border. It creates space inside the box.



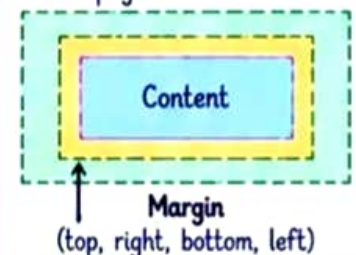
3 Border

Border is a line that surrounds the padding (and content). You can set its width, style and color.



4 Margin

Margin is the space outside the border. It separates this element from other elements on the page.

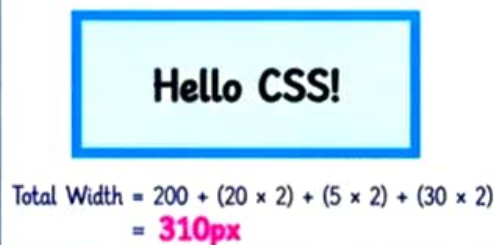


Example Code

CSS Code

```
.box {
  width: 200px;
  padding: 20px;
  border: 5px solid #3498db;
  margin: 30px;
  background-color: #e3f2fd;
}
```

Result (In Browser)



Box Model Dimensions

Total Width =

margin-left + border-left + padding-left + width + padding-right + border-right + margin-right

Total Height =

margin-top + border-top + padding-top + height + padding-bottom + border-bottom + margin-bottom

Key Takeaways

- ✓ Every element in CSS is a box.
- ✓ Content holds your actual content.
- ✓ Padding creates space inside the box.
- ✓ Border is a line around the padding.
- ✓ Margin creates space outside the box.
- ✓ Use the box model to control spacing and layout effectively.

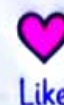
Tip:

Use the browser's DevTools (Inspect Element) to view and experiment with the box model in real time!

Inspect
Experiment
Understand
Build Better!

Codekit_

"Understand the Box, Build Better Designs."



7. CSS Width & Height

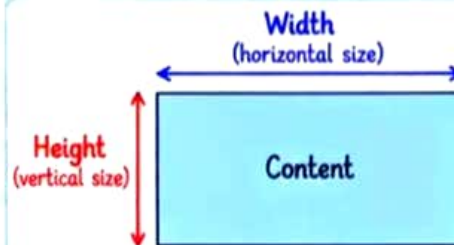
Width + Height + Max/Min Width + Max/Min Height

Learn
Build
Grow
with
Codekit_

What are Width & Height ?

- The width property sets the horizontal size (width) of an element.
- The height property sets the vertical size (height) of an element.
- You can use different units like px, %, rem, em, vw, vh, etc.
- Width and height help you control the size of elements on a web page.

Visual Representation



Control
Size
Create
Better
Layouts!

Width = Left to Right
Height = Top to Bottom

1 Width (Fixed Value)

Set a specific width using pixels (px).

CSS Code

```
.box {
  width: 300px;
  background-color: #4f8ef7;
  color: white;
}
```

Result (In Browser)

Width: 300px

2 Height (Fixed Value)

Set a specific height using pixels (px).

CSS Code

```
.box {
  height: 150px;
  background-color: #10b981;
  color: white;
}
```

Result (In Browser)

Height:
150px

3 Width & Height Together

Set both width and height for an element.

CSS Code

```
.box {
  width: 250px;
  height: 120px;
  background-color: #f97316;
  color: white;
}
```

Result (In Browser)

Width: 250px
Height: 120px

4 Using Percentage (%)

Set width in percentage (responsive).

CSS Code

```
.box {
  width: 80%;
  height: 100px;
  background-color: #8b5cf6;
  color: white;
}
```

Result (In Browser)

Width: 80%
(Responsive)
Height: 100px

5 Max-Width & Min-Width

Limit the maximum or minimum width.

CSS Code

```
.box {
  width: 80%;
  max-width: 500px; /* never exceed */
  min-width: 200px; /* never go below */
  background-color: #06b6d4;
}
```

Result (In Browser)

Min-Width: 200px
Max-Width: 500px

6 Max-Height & Min-Height

Limit the maximum or minimum height.

CSS Code

```
.box {
  height: 50vh;
  max-height: 400px; /* never exceed */
  min-height: 150px; /* never go below */
  background-color: #ef4444;
}
```

Result (In Browser)

Min-Height: 150px
Max-Height: 400px

7 Common Units for Size

You can use different units to set width and height.

Unit	Meaning	Example
px	Pixels (fixed size)	width: 300px;
%	Percentage (relative)	width: 80%;
em	Relative to parent font size	width: 10em;
rem	Relative to root font size	width: 10rem;
vw	Viewport width (1vw = 1% of screen)	width: 50vw;
vh	Viewport height (1vh = 1% of screen)	height: 50vh;

Key Takeaways

- ✓ width sets the horizontal size.
- ✓ height sets the vertical size.
- ✓ Use max-width and min-width to control width limits.
- ✓ Use max-height and min-height to control height limits.
- ✓ Use % or viewport units for responsive design.

Tip:

Use relative units (% , vw , vh) for responsive websites and combine with max/min values for better flexibility.



Responsive
Design
Starts
Here!

Codekit_

"Right Size. Better Layouts. Beautiful Websites."



Like



Comment



Share



Save

8. CSS Margin & Padding

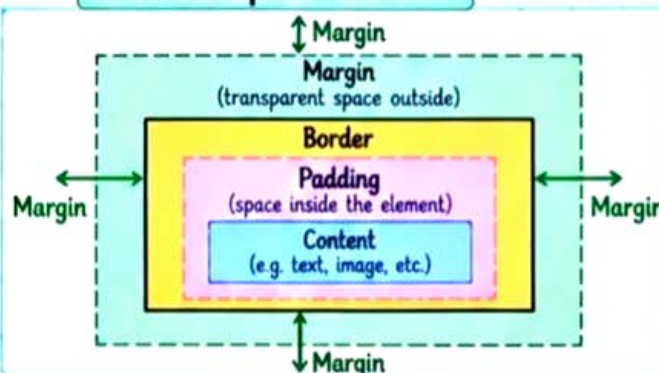
Spacing + Shorthand Properties + Practical Examples

Learn
Build
Grow
with
Codekit_

What are Margin & Padding ?

- Margin and padding are used to create space around elements.
- Padding adds space inside an element (between content and border).
- Margin adds space outside an element (between the element and other elements).
- They help you control the layout and make your website look clean and readable.

Visual Representation



Margin
Pushes
Elements
Away
Padding
Gives Space
Inside!

1 CSS Margin (Spacing Outside)

Margin creates space outside the element.

CSS Code

```
.box {
  margin: 20px;
  background-color: #4f8ef7;
  color: white;
}
```

Output (In Browser)



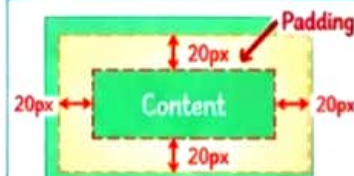
2 CSS Padding (Spacing Inside)

Padding creates space inside the element.

CSS Code

```
.box {
  padding: 20px;
  background-color: #50c878;
  color: white;
}
```

Output (In Browser)



3 Shorthand Properties

You can set top, right, bottom and left values in one line.

Property	Values	Meaning
margin: 10px;	all sides	10px on all sides
margin: 10px 20px;	top/bottom 10px left/right 20px	2 values (vertical horizontal)
margin: 10px 20px 30px;	top 10px left/right 20px bottom 30px	3 values (top horizontal bottom)
margin: 10px 20px 30px 40px;	top 10px right 20px bottom 30px left 40px	4 values (top right bottom left)

Same pattern works for padding:
padding: 10px 20px 30px 40px;

4 Practical Examples

Let's see margin and padding in action.

CSS Code

```
.card {
  margin: 30px;
  padding: 20px;
  border: 2px solid #333;
  background-color: #fff8dc;
  border-radius: 8px;
}
```

Output (In Browser)

This is a card.
It has margin, padding,
border and a background color.

5 Real-World Example (Button)

CSS Code

```
.btn {
  margin: 10px 0;
  padding: 12px 24px;
  background-color: #007bff;
  color: white;
  border: none;
  border-radius: 6px;
  cursor: pointer;
}
```

Output (In Browser)



Key Takeaways

- ✓ Margin creates space outside an element.
- ✓ Padding creates space inside an element.
- ✓ Use shorthand properties to save time.
- ✓ Margin and padding help in better spacing and layout.
- ✓ Experiment with different values to see the effect.

Tip:

Use browser DevTools (Inspect Element) to adjust margin and padding in real time and understand how they work!



Small
Spacing
Makes a
Big Difference!

Codekit_

"Clean Spacing. Beautiful Designs."



Like



Comment



Share



Save

9. CSS Borders & Border Radius

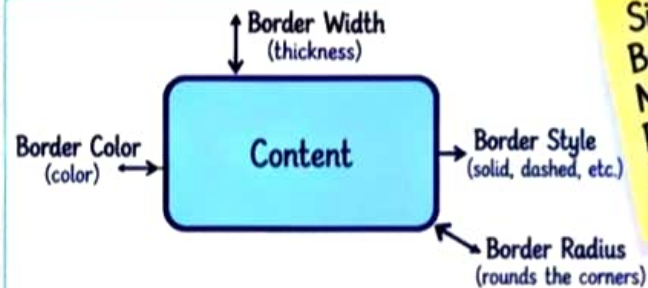
Border Style + Width + Color + Rounded Corners

Learn
Build
Grow
with
Codekit_

What are Borders & Border Radius ?

- Borders add a visible outline around an element.
- You can customize the border style, width and color.
- Border radius is used to round the corners of an element.
- They help make your website look clean, modern and attractive.

Anatomy of a Border



Simple
Borders
Make a
Big
Difference!

1 Border Style

You can set different border styles.

CSS Code

```
.box {
  border-style: solid;
  border-width: 2px;
  border-color: #3498db;
}
```

Output (In Browser)

solid dashed dotted

double groove ridge

inset outset none

2 Border Width

You can set the thickness of the border.

CSS Code

```
.box {
  border-width: 1px; /* thin */
  /* or use */
  border-width: 5px; /* thick */
  /* or individual sides */
  border-top-width: 3px;
  border-right-width: 6px;
  border-bottom-width: 3px;
  border-left-width: 6px;
}
```

Output (In Browser)

1px 3px 5px

3 Border Color

You can set the color of the border using color names, HEX, or RGB.

CSS Code

```
.box {
  border: 2px solid red;
}
.box2 {
  border: 2px solid #27ae60;
}
.box3 {
  border: 2px solid rgb(155, 89, 182);
}
```

Output (In Browser)

red #27ae60 rgb(155, 89, 182)

4 Border Radius

Border radius is used to round the corners.

CSS Code

```
.box {
  border: 2px solid #3498db;
  border-radius: 10px;
}
```

Output (In Browser)

0px (square) 10px (slight round) 50px (fully round)

5 Individual Corner Radius

You can set radius for each corner separately.

CSS Code

```
.box {
  border: 2px solid #e67e22;
  border-top-left-radius: 20px;
  border-top-right-radius: 5px;
  border-bottom-right-radius: 20px;
  border-bottom-left-radius: 5px;
}
```

Output (In Browser)

Different Corner Radius

6 Shorthand Property

You can set style, width and color in one line.

CSS Code

```
.box {
  border: 2px solid #3498db;
  border-radius: 15px;
}
```

Output (In Browser)

Shorthand Example

Key Takeaways

- ✓ Borders add an outline to elements.
- ✓ You can customize border style, width and color.
- ✓ Border radius makes rounded corners.
- ✓ Use individual corner radius for unique shapes.
- ✓ Use shorthand properties to write cleaner code.

Tip:

Use border-radius: 50% to create circles (when width and height are equal). Experiment with different styles to match your design!

Style
Your Website
With
Confidence!

10. CSS Display Property

Block + Inline + Inline-Block + None

Learn
Build
Grow
with
Codekit_

What is the Display Property ?

- The display property specifies how an element is displayed on the webpage.
- It controls the layout behavior (block, inline, inline-block, etc.) of an element.
- It helps you arrange elements in different ways.
- Some common values are: block, inline, inline-block, none, flex, grid, etc.

Visual Comparison

Block	I am a block element (full width)		
Inline	Inline 1	Inline 2	Inline 3
Inline-Block	Inline-Block 1	Inline-Block 2	Inline-Block 3
None	I am hidden (display: none)		

Different
Display
Values
Different
Layouts!

1 Block (display: block)

Block elements take full width of the parent container and start on a new line.

HTML Code

```
<div class="block">
  I am a block element
</div>
```

CSS Code

```
.block {
  display: block;
  background-color: #4f8ef7;
  color: white;
  padding: 10px;
  margin: 10px 0;
}
```

Output (In Browser)

I am a block element

- ✓ Takes full width
- ✓ Starts on a new line
- ✓ Useful for layout sections like div, header, footer, etc.

2 Inline (display: inline)

Inline elements only take the required width and do not start on a new line.

HTML Code

```
<span class="inline">
  Inline 1
</span>
<span class="inline">
  Inline 2
</span>
```

CSS Code

```
.inline {
  display: inline;
  background-color: #ffb3c7;
  padding: 5px 10px;
  margin: 5px;
}
```

Output (In Browser)

Inline 1 Inline 2

- ✓ Takes only required width
- ✓ Does not start on a new line
- ✓ Commonly used with span, a, img, etc.

3 Inline-Block (display: inline-block)

Behaves like an inline element (but allows setting width, height, margin and padding).

HTML Code

```
<div class="inline-block">
  Box 1
</div>
<div class="inline-block">
  Box 2
</div>
```

CSS Code

```
.inline-block {
  display: inline-block;
  width: 120px;
  padding: 10px;
  margin: 5px;
  background-color: #b6f2c1;
  border: 1px solid #2e7d32;
  text-align: center;
}
```

Output (In Browser)

Box 1 Box 2

- ✓ Sits in the same line
- ✓ Allows width, height, margin and padding
- ✓ Useful for buttons, cards, etc.

4 None (display: none)

Hides the element completely. It does not take any space on the page.

HTML Code

```
<div class="hidden">
  I am hidden
</div>
<p>This text is visible.</p>
```

CSS Code

```
.hidden {
  display: none;
}
```

Output (In Browser)

This text is visible.

- ✓ Element is completely hidden
- ✓ Does not take any space
- ✓ Useful for toggling elements with JavaScript

Key Takeaways

- ✓ display controls how an element is shown on the page.
- ✓ block = full width, new line.
- ✓ inline = only required width.
- ✓ inline-block = inline behavior + allow width/height.
- ✓ none = hidden, no space.
- ✓ Other useful values: flex, grid, etc.

Tip:

Use the browser's DevTools (Inspect Element) to try different display values and see how the layout changes in real time!



Play
Experiment
Learn
Build Better!

Codekit_

"Better CSS. Better Layouts. Better Websites."



Like



Comment



Share



Save

11. CSS Position Property

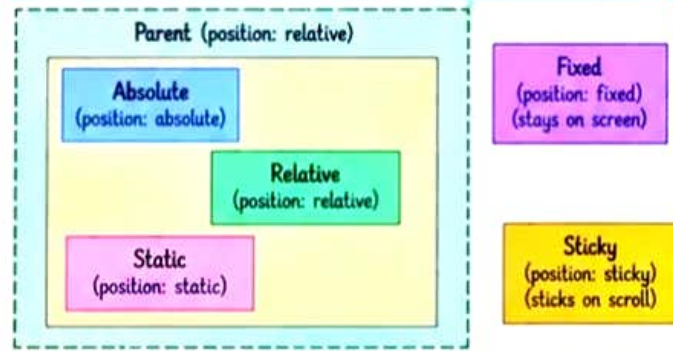
Static + Relative + Absolute + Fixed + Sticky

Learn
Build
Grow
with
Codekit_

What is the Position Property ?

- The position property specifies how an element is positioned on a web page.
- It works with top, right, bottom and left properties to move elements.
- It controls the element's positioning method (static, relative, absolute, fixed, or sticky).
- Each value behaves differently and is useful in different situations.

Positioning Reference



Position
Elements
Exactly
Where
You Want !

1 Static (default)	2 Relative	3 Absolute	4 Fixed	5 Sticky
Elements are positioned normally in the document flow. top, right, bottom, left have no effect.	Element is positioned relative to its normal position. It can be moved using top, right, bottom, left.	Element is positioned relative to the nearest positioned ancestor (parent with position relative, absolute or fixed). If no ancestor, it uses <body>.	Element is positioned relative to the viewport (browser window). It stays in the same place even when you scroll.	Element behaves like relative until it reaches a scroll position, then it sticks like fixed.
HTML + CSS <pre><div class="static"> I am static </div> .static { position: static; background: #93c5fd; }</pre>	HTML + CSS <pre><div class="relative"> I am relative </div> .relative { position: relative; top: 20px; left: 20px; background: #fca5a5; }</pre>	HTML + CSS <pre><div class="parent"> <div class="absolute"> I am absolute </div> </div> .parent { position: relative; height: 120px; background: #e5e7eb; } .absolute { position: absolute; top: 10px; right: 10px; background: #34d399; }</pre>	HTML + CSS <pre><div class="fixed"> I am fixed </div> .fixed { position: fixed; bottom: 20px; right: 20px; background: #fbbf24; }</pre>	HTML + CSS <pre><div class="sticky"> I am sticky </div> .sticky { position: sticky; top: 0; background: #c084fc; padding: 10px; }</pre>
Output (In Browser)	Output (In Browser)	Output (In Browser)	Output (In Browser)	Output (In Browser)
- Default value - Follows normal flow - Cannot be moved using top, right, bottom, left	- Moves from original position - Space is still reserved - Useful for small adjustments	- Removed from normal flow - Positioned inside nearest positioned parent - Useful for dropdowns, tooltips, modals, etc.	- Removed from normal flow - Stays in the same position on scroll - Useful for chat buttons, back-to-top, etc.	- Works on scroll - Sticks to a defined position - Useful for headers, navigation bars, etc.

Key Takeaways

- position controls how an element is placed on the page.
- Use top, right, bottom, left to move elements.
- Choose the right value based on your layout needs.
- relative, absolute, fixed and sticky help in modern web designs.

Tip:

Use position with z-index when elements overlap (to control which one appears on top).



Position
Smart.
Design
Better!

Codekit_

"Master CSS. Build Better Websites."



Like



Comment



Share



Save

12. CSS Flexbox Basics

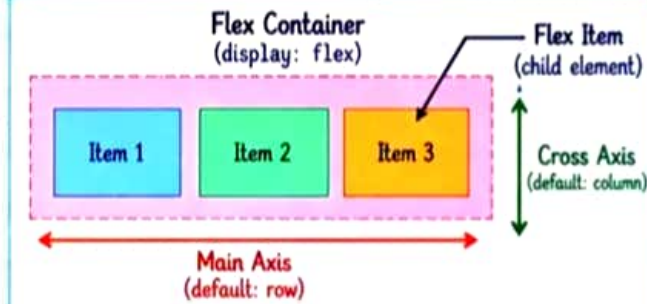
Flex Container + Direction + Justify Content + Align Items

Learn
Build
Grow
with
Codekit_

What is Flexbox ?

- Flexbox (Flexible Box) is a one-dimensional layout model in CSS.
- It helps you align and distribute space among elements easily.
- It works with a flex container (parent) and flex items (children).
- It makes it simple to create responsive and modern layouts.

Flexbox Terminology



Simple
Layouts
Powerful
Designs
with
Flexbox !

1 Flex Container

Use display: flex on the parent element.

CSS Code

```
.container {
  display: flex;
  background: #e3f2fd;
  padding: 10px;
  gap: 10px;
}
```

Output (In Browser)



- ✓ Makes the container a flexbox.
- ✓ All direct children become flex items.
- ✓ You can also use inline-flex.

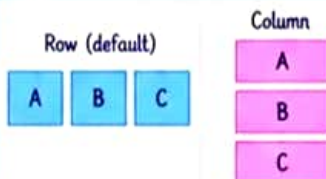
2 Flex Direction

Defines the direction of flex items (row or column).

CSS Code

```
.row { display: flex;
      flex-direction: row; }
.column { display: flex;
         flex-direction: column; }
```

Output (In Browser)



- ✓ row → left to right (default)
- ✓ column → top to bottom
- ✓ Use based on your layout need.

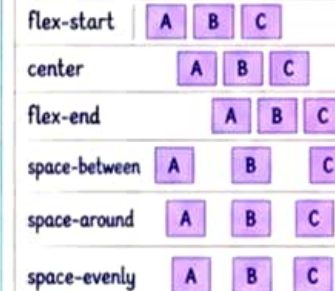
3 Justify Content

Aligns items along the main axis (horizontally in a row).

CSS Code

```
.container {
  display: flex;
  justify-content: center; }
/* Options: flex-start, center,
flex-end, space-between,
space-around, space-evenly */
```

Output Examples



4 Align Items

Aligns items along the cross axis (vertically in a row).

CSS Code

```
.container {
  display: flex;
  align-items: center; }
/* Options: stretch (default),
flex-start, center,
flex-end, baseline */
```

Output Examples



5 Flex Shorthand

You can use the flex shorthand property on items.

CSS Code

```
.item {
  flex: 1; /* grows equally */
}
.item2 {
  flex: 2; /* takes 2x space */
}
```

6 Practical Example

A complete flexbox navbar example.

CSS Code

```
.navbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  background: #1e3a8a;
  padding: 10px 20px;
  color: white;
}
.logo { font-size: 20px; font-weight: bold; }
.menu { display: flex; gap: 20px; }
.menu a { color: white; text-decoration: none; }
```

Output (In Browser)



Key Takeaways

- ✓ Use display: flex on the parent.
- ✓ flex-direction controls the direction.
- ✓ justify-content aligns items on the main axis.
- ✓ align-items aligns items on the cross axis.
- ✓ Flexbox makes responsive layouts simple and powerful.

Tip: Combine justify-content and align-items to perfectly center elements (both horizontally & vertically).

💡 Flexbox
= Flexible
Layouts
= Happier You!

13. CSS Flexbox Advanced

Wrap + Gap + Flex Grow + Flex Shrink + Order

Learn
Build
Grow
with
Codekit_

Why Use Advanced Flexbox ?

- Helps you build responsive and dynamic layouts.
- Gives more control over how items behave inside a flex container.
- Useful for real-world UI components like cards, navbars, dashboards, etc.
- Properties like wrap, gap, grow, shrink and order make your designs more flexible and powerful.

Flexbox Advanced at a Glance

flex-wrap
(wrap items)

gap
(space between)

flex-grow
(allow item to grow)

flex-shrink
(allow item to shrink)

order
(change the order of items)

More
Control
More
Flexible
Layouts !

1 Flex Wrap

Allows flex items to wrap into multiple lines if there is not enough space.

CSS Code

```
.container {
  display: flex;
  flex-wrap: wrap;
  gap: 10px;
}
```

Output (In Browser)



- Items automatically wrap to the next line when they don't fit.

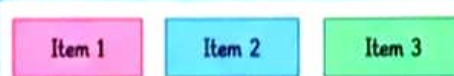
2 Gap

Adds space between flex items (row and column).

CSS Code

```
.container {
  display: flex;
  gap: 20px; /* space between items */
}
```

Output (In Browser)



- Adds consistent space between items.
- Works for both row and column direction.
- Cleaner than using margin.

3 Flex Grow

Defines how much a flex item should grow relative to others.

CSS Code

```
.item1 { flex-grow: 1; }
.item2 { flex-grow: 2; }
.item3 { flex-grow: 1; }
```

Output (In Browser)



- Item 2 takes 2x more space than Items 1 and 3.
- Default value is 0 (no grow).

4 Flex Shrink

Defines how much a flex item should shrink if there is not enough space.

CSS Code

```
.item1 { flex-shrink: 1; }
.item2 { flex-shrink: 2; }
.item3 { flex-shrink: 1; }
```

Output (In Browser)



- Item 2 shrinks 2x faster than others.
- Default value is 1 (items can shrink).

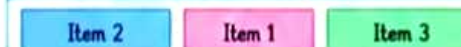
5 Order

Changes the visual order of flex items without changing the HTML.

CSS Code

```
.item1 { order: 2; }
.item2 { order: 1; }
.item3 { order: 3; }
```

Output (In Browser)



- Items appear in the specified order.
- Useful for responsive design (e.g., changing order on mobile).

Real-World Example

Codekit_ Home About Services Contact

CSS Code (Navbar)

```
.navbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  flex-wrap: wrap;
  gap: 20px;
}
```

Key Takeaways

- flex-wrap helps items move to the next line.
- gap adds space between items.
- flex-grow allows items to take extra space.
- flex-shrink allows items to reduce space.
- order changes the visual order of items.

Tip:

Combine these properties to build fully responsive and flexible layouts for real-world projects!

Flexible
Layouts
= Better
Websites!

Codekit_

"Flex Smarter. Build Better."



Like



Comment



Share



Save

14. CSS Grid Basics

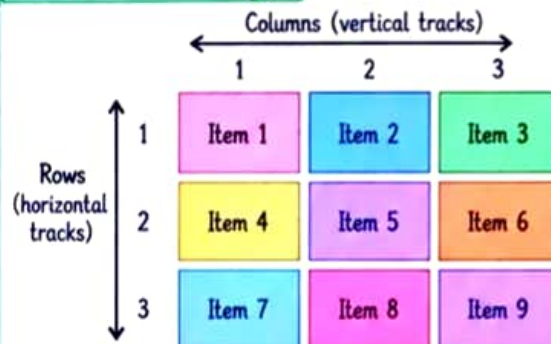
Grid Container + Rows + Columns + Gap

Learn
Build
Grow
with
Codekit_

What is CSS Grid ?

- CSS Grid is a two-dimensional layout system in CSS.
- It helps you create complex and responsive layouts easily.
- You can control rows, columns, gaps and placement of items.
- It is perfect for modern web layouts like dashboards, galleries, and card sections.

Grid Terminology



Simple
Grids
Powerful
Layouts
Modern
Web Design !

1 Grid Container

Use display: grid to create a grid container.

CSS Code

```
.grid-container {
  display: grid;
  background: #f0f0f0;
  padding: 10px;
}
```

Output (In Browser)



- ✓ display: grid makes the element a grid container.
- ✓ All direct children become grid items.

2 Grid Columns

Use grid-template-columns to define the number and size of columns.

CSS Code

```
.grid-container {
  display: grid;
  grid-template-columns:
    1fr 1fr 1fr; /* 3 equal
    columns */
  gap: 10px;
}
```

Output (In Browser)



- ✓ Creates 3 equal columns.
- ✓ You can also use fixed values like 200px 200px 200px or repeat(3, 1fr).

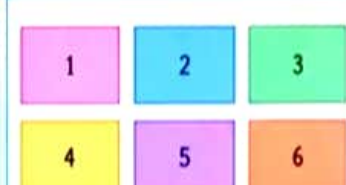
3 Grid Rows

Use grid-template-rows to define the number and size of rows.

CSS Code

```
.grid-container {
  display: grid;
  grid-template-rows:
    100px 100px; /* 2 rows
    with 100px height */
  gap: 10px;
}
```

Output (In Browser)



- ✓ Creates 2 rows with fixed height.
- ✓ You can also use 1fr, auto or minmax().

4 Gap

Use gap to set space between rows and columns.

CSS Code

```
.grid-container {
  display: grid;
  grid-template-columns:
    repeat(3, 1fr);
  grid-template-rows:
    100px 100px;
  gap: 20px; /* space between
    rows and columns */
}
```

Output (In Browser)



- ✓ gap sets both row and column spacing.
- ✓ You can also use row-gap and column-gap separately.

Key Takeaways

- ✓ Use display: grid to create a grid.
- ✓ Use grid-template-columns to define columns.
- ✓ Use grid-template-rows to define rows.
- ✓ Use gap to set space between rows and columns.
- ✓ Grid makes complex layouts simple and responsive.

Tip:

Use fr (fractional unit) for flexible layouts that adapt to different screen sizes!

Think
In Grids
Design
Without Limits!

Codekit_

"Grids Today. Better Layouts Tomorrow."



15. CSS Grid Advanced

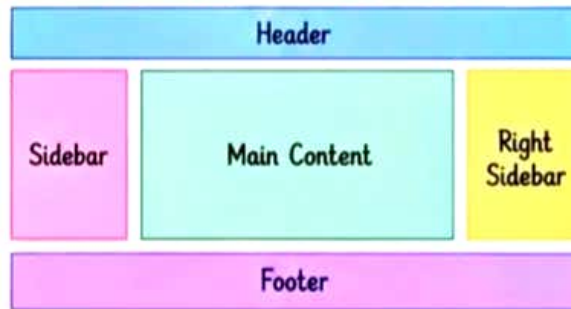
Grid Areas + Span + Auto-fit + Minmax + Responsive Grid

Learn
Build
Grow
with
Codekit_

Why Learn Advanced Grid ?

- Advanced Grid gives you more control to create complex and responsive layouts.
- Grid areas make layouts easy to manage.
- Span lets items take multiple rows/columns.
- auto-fit and minmax help create flexible and responsive grids.
- Perfect for modern web designs like dashboards, landing pages, and card layouts.

Example Layout (Using Grid Areas)



Advanced
Grids
Modern
Layouts
Real-World
Designs !

1

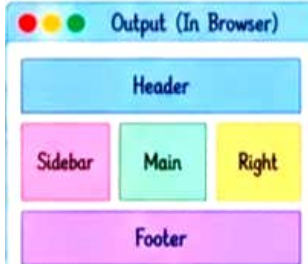
Grid Areas

Use grid-template-areas to create named layout sections.

CSS Code

```

.container {
  display: grid;
  grid-template-areas:
    "header header header"
    "sidebar main right"
    "footer footer footer";
  gap: 10px;
}
```



- ✓ Makes layouts easier to understand.
- ✓ Perfect for complex page structures.
- ✓ Reusable and maintainable.

2

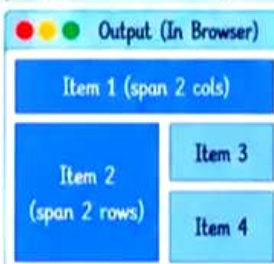
Span

Use grid-column and grid-row to make items span multiple columns or rows.

CSS Code

```

.item {
  background: #4f8ef7;
  color: white;
  padding: 20px;
}
.item1 {
  grid-column: 1 / 3;
}
.item2 {
  grid-row: 2 / 4;
```



- ✓ grid-column controls width (span columns).
- ✓ grid-row controls height (span rows).
- ✓ Useful for highlighting important content.

3

Auto-fit

Use repeat(auto-fit, minmax()) to create responsive grids that automatically adjust to screen size.

CSS Code

```

.grid {
  display: grid;
  grid-template-columns:
    repeat(auto-fit,
      minmax(200px, 1fr));
  gap: 20px;
}
```



- ✓ Automatically fits as many columns as possible.
- ✓ Items wrap to next line when needed.
- ✓ Great for responsive card layouts.

4

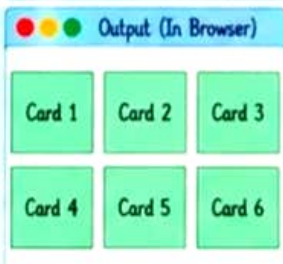
Minmax

Use minmax() to set a minimum and maximum size for grid items.

CSS Code

```

.grid {
  display: grid;
  grid-template-columns:
    repeat(3, minmax(
      150px, 1fr));
  gap: 15px;
}
```



- ✓ minmax(min, max) sets flexible size limits.
- ✓ Prevents items from becoming too small or too large.
- ✓ Useful for fluid, modern layouts.

5

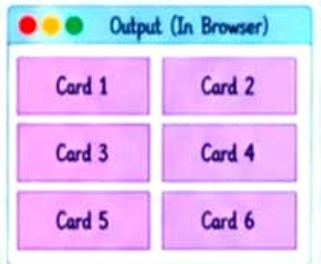
Responsive Grid

Combine auto-fit and minmax() for a fully responsive layout.

CSS Code

```

.grid {
  display: grid;
  grid-template-columns:
    repeat(auto-fit,
      minmax(250px, 1fr));
  gap: 20px;
}
```



- ✓ Automatically adjusts based on screen size.
- ✓ Works great on mobile, tablet and desktop.
- ✓ Most commonly used in modern websites.

Key Takeaways

- ✓ Use grid-template-areas for structured layouts.
- ✓ Use grid-column and grid-row to span items.
- ✓ Use auto-fit to create responsive grids.
- ✓ Use minmax() for flexible sizes.
- ✓ Combine these techniques for modern, responsive designs.

Tip:

Use auto-fit + minmax() for responsive card layouts. It automatically adjusts columns as the screen size changes!



Grid
Smarter.
Design
Bigger !

Codekit_

"Advanced Grids. Limitless Layouts."



Like



Comment



Share



Save

16. CSS Pseudo-Classes & Pseudo-Elements

:hover + :focus + :nth-child() + ::before + ::after

Learn
Build
Grow
with
Codekit

What are Pseudo-Classes & Pseudo-Elements ?

- Pseudo-classes select elements based on a special state (like hover, focus, position, etc.).
- Pseudo-elements let you style a specific part of an element (like before or after the content).
- They help you create interactive and attractive web designs without extra HTML.
- Commonly used in modern, responsive websites for better UI/UX.

Pseudo-Classes vs Pseudo-Elements

Pseudo-Classes

- Select elements in a special state.
- Use single colon (:)
- Examples: :hover, :focus, :nth-child()
- Change style based on user action or position.

Pseudo-Elements

- Style a specific part of element.
- Use double colon (::)
- Examples: ::before, ::after
- Used to insert content or style a part of an element without extra HTML.

Small
Selectors
Big Impact
Make
Beautiful
Designs !

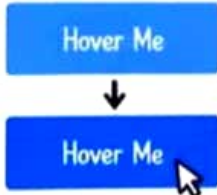
1 :hover

Applies styles when you hover over an element with your mouse.

CSS Code

```
.btn:hover {
  background-color: #2563eb;
  color: white;
  transform: scale(1.05);
}
```

Output (In Browser)



- ✓ Useful for interactive buttons, links, cards, etc.
- ✓ Improves user experience with visual feedback.

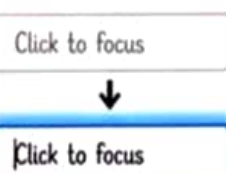
2 :focus

Applies styles when an element gets focus (e.g., input field).

CSS Code

```
input:focus {
  border: 2px solid #2563eb;
  outline: none;
  box-shadow: 0 0 5px
    rgba(37, 99, 235, 0.5);
}
```

Output (In Browser)



- ✓ Useful for forms and accessibility.
- ✓ Helps users identify the active element.

3 :nth-child()

Selects an element based on its position among its siblings.

CSS Code

```
li:nth-child(odd) {
  background-color: #f1f5f9;
}
li:nth-child(even) {
  background-color: #e0e7ff;
}
```

Output (In Browser)

- Item 1
- Item 2
- Item 3
- Item 4
- Item 5

- ✓ Useful for styling lists, tables, or grids.
- ✓ Can also use formulas like nth-child(3), nth-child(2n), etc.

4 ::before

Insert content before the actual content of an element.

CSS Code

```
.note::before {
  content: "★";
  color: #f59e0b;
  font-size: 20px;
  margin-right: 8px;
}
```

Output (In Browser)



- ✓ Adds decorative content before an element.
- ✓ Commonly used for icons, labels, quotes, etc.

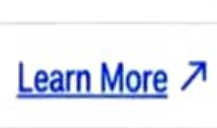
5 ::after

Insert content after the actual content of an element.

CSS Code

```
.link::after {
  content: "➔";
  color: #2563eb;
  margin-left: 6px;
  font-size: 14px;
}
```

Output (In Browser)



- ✓ Adds decorative content after an element.
- ✓ Often used for link icons, read more arrows, etc.

Key Takeaways

- ✓ Pseudo-classes help you style elements based on their state.
- ✓ Pseudo-elements let you add and style content without extra HTML.
- ✓ :hover, :focus, :nth-child() improve interactivity and layout control.
- ✓ ::before and ::after are great for decorative content.
- ✓ These selectors make your website more modern and user-friendly.

Tip:

You can combine pseudo-classes and pseudo-elements too!
Example: a:hover::after { ... }



Style
Smarter.
Create
Better
Experiences!

Codekit

"Small Selectors. Big Possibilities."



Like



Comment



Share



Save

17. CSS Transitions & Transforms

Transition + Translate + Rotate + Scale + Skew

Learn
Build
Grow
with
Codekit_

Why Use Transitions & Transforms ?

- Transitions let you smoothly change CSS properties from one value to another.
- Transforms let you move, rotate, scale or skew elements.
- They make your website more interactive, modern and visually appealing.
- Commonly used in buttons, cards, images, menus and animations.

What's the Difference ?

Transition

- Smoothly changes a CSS property over a given time.
- Needs a trigger (like :hover).
- Works with many properties such as color, background, transform, etc.

Transform

- Changes the position, size, rotation or shape of an element.
- Does not need a trigger (but often used with :hover).
- More performant (uses GPU).
- Common transforms: translate, rotate, scale, skew.

Smooth
Effects
Modern UI
Better
User
Experience !

1 Transition (Hover Effect)

Smoothly changes a property when you hover over the element.

CSS Code

```
.btn {
  background: #2563eb;
  color: white;
  padding: 10px 20px;
  border: none;
  transition: all 0.3s ease;
}
.btn:hover {
  background: #1e40af;
  transform: scale(1.05);
}
```

Output (In Browser)

Hover Me

- ✓ Smooth transition effect on hover.

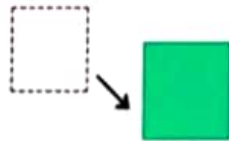
2 Translate (Move)

Moves an element along X or Y axis.

CSS Code

```
.box {
  width: 100px;
  height: 100px;
  background: #10b981;
  transition: transform 0.3s ease;
}
.box:hover {
  transform: translate(50px, 20px);
}
```

Output (In Browser)



- ✓ Moves the element 50px right and 20px down.

3 Rotate

Rotates an element by a given degree.

CSS Code

```
.box {
  width: 100px;
  height: 100px;
  background: #f59e0b;
  transition: transform 0.3s ease;
}
.box:hover {
  transform: rotate(45deg);
}
```

Output (In Browser)



- ✓ Rotates the element by 45 degrees.

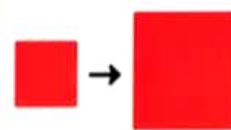
4 Scale

Increases or decreases the size of an element.

CSS Code

```
.box {
  width: 100px;
  height: 100px;
  background: #ef9444;
  transition: transform 0.3s ease;
}
.box:hover {
  transform: scale(1.5);
}
```

Output (In Browser)



- ✓ Scales the element to 1.5 times its size.

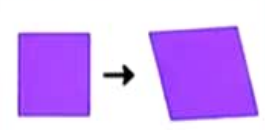
5 Skew

Tilts an element along X or Y axis.

CSS Code

```
.box {
  width: 100px;
  height: 100px;
  background: #8b5cf6;
  transition: transform 0.3s ease;
}
.box:hover {
  transform: skew(20deg, 10deg);
}
```

Output (In Browser)



- ✓ Skews the element by 20deg (X) and 10deg (Y).

Key Takeaways

- ✓ Use transition for smooth changes.
- ✓ Use transform to move, rotate, scale or skew elements.
- ✓ Common transform functions: translate(), rotate(), scale(), skew().
- ✓ Transitions make UI feel more smooth and professional.
- ✓ Transforms are lightweight and improve performance.

Tip:

You can combine multiple transforms:

```
.box {
  transform: translate(50px, 20px)
            rotate(45deg) scale(1.2);
}
```



Animate
Your Ideas
with
CSS !

Codekit_

"Smooth Transitions. Stunning Transformations."



Like



Comment



Share



Save

18. CSS Animations

@keyframes + Animation Name + Duration + Delay + Infinite Animation

Learn
Build
Grow
with
Codekit_

What are CSS Animations ?

- CSS Animations let you animate elements without JavaScript.
- You define a set of styles using @keyframes and apply them with animation properties.
- You can control the animation name, duration, delay, iteration count (infinite), timing function and more.
- Great for creating modern, interactive and engaging UI/UX.

Key Animation Properties

Property	Description
@keyframes	Defines the animation steps (from and to).
animation-name	Specifies the name of the animation.
animation-duration	How long the animation takes.
animation-delay	When the animation should start.
animation-iteration-count	How many times to repeat (e.g. infinite).
animation-timing-function	Speed curve (e.g. ease, linear, etc.).

Animate
Your Ideas
Make
Websites
Come
Alive !

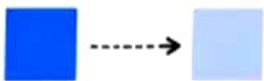
1 Basic Animation (@keyframes)

A simple animation that moves a box from left to right.

```

CSS Code
@keyframes moveRight {
  from {
    transform: translateX(0);
  }
  to {
    transform: translateX(200px);
  }
}
.box {
  width: 50px;
  height: 50px;
  background: #2563eb;
  animation-name: moveRight;
  animation-duration: 2s;
}
```

Output (In Browser)



- ✓ Box moves from left to right.

2 Animation Delay

Starts the animation after a delay.

```

CSS Code
@keyframes fadeIn {
  from {
    opacity: 0;
  }
  to {
    opacity: 1;
  }
}
.box {
  width: 80px;
  height: 80px;
  background: #10b981;
  animation: fadeIn 2s;
  animation-delay: 1s;
}
```

Output (In Browser)



- ✓ Fades in after 1 second.

3 Infinite Animation

Repeats the animation forever.

```

CSS Code
@keyframes bounce {
  0%, 100% {
    transform: translateY(0);
  }
  50% {
    transform: translateY(-50px);
  }
}
.box {
  width: 50px;
  height: 50px;
  background: #ef4444;
  animation: bounce 1s
    ease-in-out infinite;
}
```

Output (In Browser)



- ✓ Box bounces up and down infinitely.

4 Rotate Animation

Continuously rotates the element.

```

CSS Code
@keyframes spin {
  from {
    transform: rotate(0deg);
  }
  to {
    transform: rotate(360deg);
  }
}
.box {
  width: 60px;
  height: 60px;
  background: #f59e0b;
  animation: spin 2s
    linear infinite;
}
```

Output (In Browser)



- ✓ Box rotates continuously.

5 Combined Example

Uses multiple animation properties together.

```

CSS Code
@keyframes pulse {
  0%, 100% {
    transform: scale(1);
    background: #8b5cf6;
  }
  50% {
    transform: scale(1.2);
    background: #a78bfa;
  }
}
.box {
  width: 80px;
  height: 80px;
  border-radius: 10px;
  animation: pulse 2s
    ease-in-out 0.5s infinite;
}
```

Output (In Browser)



- ✓ Box smoothly grows and shrinks with a delay.

Key Takeaways

- ✓ Use @keyframes to define animation steps.
- ✓ Control animation with properties like name, duration, delay, etc.
- ✓ Use infinite to repeat animations.
- ✓ Animations make your website more interactive and visually appealing.
- ✓ Combine multiple properties for advanced effects.

Tip:

You can also use shorthand animation property:

```

.box {
  animation: bounce 2s ease 0.5s infinite;
}
```

Small
Animations
Big User
Experience !

Codekit_

"Code It. Animate It. Bring Ideas to Life."



Like



Comment



Share



Save

19. CSS Responsive Design

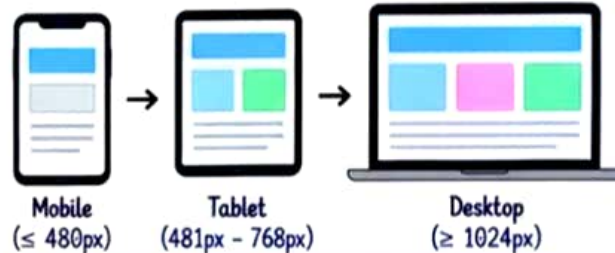
Media Queries + Mobile-First + Breakpoints + Responsive Layouts

Learn
Build.
Grow
with
Codekit_

What is Responsive Design ?

- Responsive Design makes your website look good on all devices (mobile, tablet, desktop).
- It automatically adjusts the layout, size and content based on the screen size.
- We use media queries, flexible units and responsive layouts to achieve this.
- It provides a better user experience and works for all screen sizes.

Responsive Design in Action



Same Website → Different Screens → Better Experience !

Responsive
Design
Better
User
Experience
for Everyone !

1 Media Queries

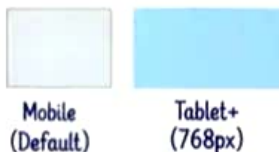
Media queries let you apply different CSS styles based on screen size, device type or other conditions.

CSS Code

```
/* Mobile (default) */
body {
  background: #f9f9f9;
}

/* Tablet and above */
@media (min-width: 768px) {
  body {
    background: #e0f2fe;
  }
}
```

Output (In Browser)



- Apply styles based on screen size.

2 Mobile-First Approach

Start by designing for small screens (mobile) first, then add styles for larger screens.

CSS Code

```
/* Base styles for mobile */
.container {
  padding: 10px;
  font-size: 14px;
}

/* Larger screens */
@media (min-width: 768px) {
  .container {
    padding: 20px;
    font-size: 16px;
  }
}
```

Output (In Browser)



- Better performance
- Cleaner code
- Easier to maintain
- Works for all devices

3 Common Breakpoints

Breakpoints are specific screen widths where you change the layout.

Device Type	Width
Extra Small (Mobile)	≤ 480px
Small (Mobile Large)	481px - 767px
Medium (Tablet)	768px - 1023px
Large (Desktop)	1024px - 1440px
Extra Large (Large Screens)	≥ 1441px

Example (Min-Width)

```
@media (min-width: 1024px) {
  .sidebar {
    display: block;
  }
}
```

- Use standard breakpoints for consistent design.

4 Responsive Layouts

Use flexible units (% , vw , rem), flexbox or grid to create responsive layouts.

CSS Code

```
.container {
  display: flex;
  flex-wrap: wrap;
  gap: 16px;
}

.card {
  flex: 1 1 250px; /* grows and shrinks */
  min-width: 250px;
  background: white;
  padding: 16px;
  border-radius: 8px;
}
```

Output (In Browser)



- Layouts adapt automatically for different screen sizes.

Key Takeaways

- Use media queries to apply styles based on screen size.
- Follow mobile-first approach.
- Use common breakpoints (480px, 768px, 1024px, 1440px).
- Use flexible units and responsive layouts (flexbox/grid).
- Test your website on multiple devices.

Tip:

Use relative units like %, rem, em, vw instead of fixed px for better responsiveness.

```
.heading {
  font-size: 2rem;
}
```



Design
Once
Works
Everywhere !



Like



Comment



Share



Save

20. CSS3 Best Practices

Variables + calc() + clamp() + Clean CSS + Accessibility + Performance

Learn
Build
Grow
with
Codekit_

Why Follow Best Practices ?

- Makes your CSS clean, consistent and easy to maintain.
- Improves website performance and loading speed.
- Helps create responsive and accessible designs for all users.
- Follows modern standards used in real world web development.
- Results in better user experience and professional code.

What We'll Cover

- ✓ CSS Variables (Custom Properties)
- ✓ Using calc() for flexible values
- ✓ Using clamp() for responsive sizing
- ✓ Writing clean and maintainable CSS
- ✓ Accessibility best practices
- ✓ Performance optimization tips

"Clean CSS today,
better websites tomorrow."

Before vs After (Clean CSS)

Before

```
.btn {
  background-color: #007bff;
  color: white;
  padding: 10px 20px;
  border: none;
  border-radius: 4px;
  font-size: 16px;
}
```

Works, but hard to maintain
and not reusable.

After (Clean & Reusable)

```
:root {
  --primary: #007bff;
  --white: #ffffff;
  --radius: 4px;
}

.btn {
  background: var(--primary);
  color: var(--white);
  padding: 10px 20px;
  border: none;
  border-radius: var(--radius);
  font-size: 1rem;
}
```

Clean, reusable and easy
to update.

1 CSS Variables (Custom Properties)

Store reusable values like colors, spacing, fonts, etc.

```

:root {
  --primary: #2563eb;
  --text: #1f2937;
  --bg: #f9fafb;
}

.button {
  background: var(--primary);
  color: var(--text);
}
```

Output (In Browser)

Primary Button

- ✓ Easily update values across your website.

2 Using calc()

Perform calculations for dynamic values.

```
.container {
  width: calc(100% - 40px);
  padding: 20px;
  margin: 0 auto;
}
```

Output (In Browser)

Width = 100% - 40px

- ✓ Useful for flexible and dynamic layouts.

3 Using clamp()

Create responsive sizes with minimum, preferred and maximum values.

```
h1 {
  font-size: clamp(
    1.5rem,
    4vw,
    3rem
  );
}
```

Output (In Browser)

Responsive
Heading

Scales smoothly on
all devices.

- ✓ Perfect for modern responsive design.

4 Clean CSS

Write simple, organized and maintainable code.

```
.card {
  display: flex;
  flex-direction: column;
  gap: 1rem;
  padding: 1rem;
  border-radius: 8px;
  background: #fff;
  box-shadow: 0 2px 8px rgba(0,0,0,0.1);
}
```

Output (In Browser)

Clean Code

Simple. Readable.
Maintainable.

- ✓ Use meaningful class names and consistent formatting.

5 Accessibility

Make your website usable for everyone.

```
.btn:focus {
  outline: 2px solid #2563eb;
  outline-offset: 2px;
}

a:focus, button:focus {
  outline: 2px solid #2563eb;
}

.sr-only {
  position: absolute;
  width: 1px;
  height: 1px;
  overflow: hidden;
}
```

Output (In Browser)

Focus Me

- ✓ Use proper contrast, focus states and Semantic HTML.

6 Performance

Optimize CSS for faster loading and better performance.

```

/* Good Practices */
/* 1. Use only needed CSS */
/* 2. Minify your CSS */
/* 3. Avoid !important */
/* 4. Use modern units (rem, em, clamp) */
/* 5. Use efficient selectors */
/* 6. Avoid heavy animations */
```

Output (In Browser)

Faster Loading
Better User Experience

- ✓ Smaller CSS = Faster websites.

Key Takeaways

- ✓ Use CSS variables for consistency.
- ✓ Use calc() and clamp() for flexible and responsive values.
- ✓ Write clean, organized and maintainable CSS.
- ✓ Follow accessibility guidelines for inclusive design.
- ✓ Optimize CSS for better performance and faster loading.

Tip:

Combine these features for powerful results!

```

:root {
  --size: clamp(1rem, 2vw, 2rem);
}

.card {
  padding: calc(var(--size) * 1.5);
}
```

Better
Code
Brighter
Websites!

Codekit_

"Good CSS Habits. Greater Web Experiences."



Like



Comment



Share



Save